

DaaS Engineering Take-Home

One full-stack feature slice on the DaaS stack: Postgres schema, GraphQL API, Next.js UI

DaaS
DEVELOPERS AS A SERVICE

TIME TO COMPLETE

2 days (about 4 to 5 focused hrs)

ROLE FOCUS

Full-Stack Developer

KEY STACK

Next.js 16, Apollo GraphQL, Postgres

RUNTIME

Node 24 LTS, TypeScript, Docker

1. The Project & Role

The DaaS Project is an internal stock and project-allocation tool for an AV integration business. It is a Next.js 16 portal on top of a Node GraphQL API and Postgres, deployed to AWS EKS from GitHub Actions. Over the next five sprints it grows from an inventory tool into a full project-delivery platform.

Roadmap you would be building

- **Sprint 1:** Projects, clients, contacts and vendors, kept in sync with Jetbuilt and HubSpot
- **Sprint 2:** Project products, engineering and PM release, inventory, stock adjustments, purchase orders, location and barcode tracking
- **Sprint 3:** Drawing sets with markup and measure, daily logs, punch lists
- **Sprint 4:** Schedules synced with Smartsheet, Gantt view, tasks, RMAs
- **Sprint 5:** Quoting and estimating builder that replaces Jetbuilt proposals, PDF export, change orders

We are looking for engineers comfortable with GraphQL API design, relational modelling for inventory and workflow state, and typed React with MUI and RTK Query.

2. The Challenge

Build a small standalone repo that implements **one** feature slice from the roadmap end to end: Postgres schema, GraphQL API and UI. We do not prescribe a folder structure. We do care about how your structure would scale to the sprints above, so make the boundaries visible and explain them briefly.

Suggested time split

API

Schema, migration, resolvers, one guarded mutation. About 2 hrs.

UI

List page and create form on RTK Query. About 1.5 hrs.

WRITE-UP

README and architecture note. About 45 min.

If you run out of time, ship a smaller working slice and list what you cut in the README. A finished half beats an unfinished whole.

3. Core Technical Expectations

A. Backend (Apollo Server 5 on Node, Postgres, TypeScript)

- Postgres with Prisma or Drizzle (your pick). Migrations checked in. Real foreign keys, unique constraints and check constraints where the domain calls for them.
- One folder per domain holding its `.graphql` schema and resolvers. Typed resolvers; codegen is welcome but optional.
- Our conventions: a `uuid` primary key (UUIDv7 or ULID), soft delete via `deleted_at`, timestamps on every table.
- A simple bearer token that carries a role is enough for auth. At least one mutation must enforce a role check.
- Any write that touches more than one row must run in a transaction.

B. Pick one slice

OPTION 1 · SPRINT 2

Purchase order receiving

Create a PO with line items. Receive quantities against it into a location. Each receive inserts a stock movement and updates on-hand per location. PO status (open, partial, received) is derived from quantities, never hand-set.

OPTION 2 · SPRINT 4

RMAs

Raise an RMA against a vendor with items and a reason. Status workflow Draft, Submitted, Approved, Closed with guarded transitions. Closing an RMA that returns stock adjusts on-hand.

OPTION 3 · SPRINT 1

Client & contact sync

Clients and contacts CRUD. Every create or update writes an outbox row that a small worker pushes to a mock HubSpot adapter, idempotently. Sync status is visible in the UI.

C. Frontend (Next.js 16 App Router, MUI 9, Redux Toolkit + RTK Query)

- A list page with one filter, and a create form using react-hook-form with a zod schema. Editing is optional.
- All data access through RTK Query with tag-based invalidation. Loading, empty and error states handled.
- Colours and spacing from the MUI theme, no one-off hex values.

D. Architecture note (required, half a page in the README)

Pick one and describe your design, its main trade-off, and what you would ship first:

- Making stock movements transactional and auditable across adjust, transfer, receive and allocate.
- Running Jetbuilt, HubSpot and Smartsheet syncs outside the web process across several API replicas.
- Enforcing role permissions consistently across the GraphQL API and the UI.

Bonus, only if you have time left

- Integration tests for the resolvers against a real Postgres (compose or testcontainers).
- An activity log that records who changed what.

4. Reference Documentation

Backend

- Apollo Server 5
- Prisma
- Drizzle
- Postgres transactions
- GraphQL Codegen

Frontend & integrations

- Next.js App Router
- MUI 9
- RTK Query
- react-hook-form + zod
- HubSpot CRM API

5. Submission Deliverables

1. **Code repository:** `api/` and `web/` (or a monorepo), each with a `package.json` and `.env.example`, plus a `docker-compose.yml` that starts Postgres.
 2. **README:** setup and run steps, how to seed, the architecture note from section D, and what you cut and why.
 3. **Demo artifact:** a 1 to 2 minute screen recording or a few screenshots, plus the GraphQL operations you used.
- Send a public repo link or invite us as collaborators. We read the code and commit history, so pixel-perfect design is not the goal.

6. Evaluation Rubric

AREA	WEIGHT	KEY CRITERIA
Architecture & code quality	30%	Clear domain boundaries, no copy-paste modules, typed end to end, an architecture note with a real trade-off.
Data modelling & GraphQL API	25%	Sound relational schema and constraints, our uuid and soft-delete conventions, derived state over stored flags, transactions on multi-row writes.
Frontend & UX	20%	Correct RTK Query cache handling, validated form, loading and error states, theme consistency.
Resilience	15%	Enforced role check, useful error messages from the API, idempotent worker where the slice needs one.
DX & communication	10%	Compose plus seed gets a reviewer running in minutes, readable README, clean commits.
Bonus	+15%	Integration tests, activity log.